GKP Decoding under Correlated Displacement Noise: A Bayesian and Machine-Learning Comparison

Experiment report (branch `experiment/decoder-comparison`)

November 2026

Abstract

We compare three decoders for the square GKP code under temporally correlated (AR(1)) displacement noise with Gaussian measurement noise, in a validated simplified simulator of the position quadrature:

- a memoryless nearest-lattice (NL) decoder;
- a sequential Bayesian grid filter;
- an LSTM trained on simulated syndrome histories.

Restricting corrections to the syndrome plus a lattice shift makes the syndromes independent of the decoder. Decoding then reduces to estimating a running parity, which allows paired comparisons on common noise and supervised training without distribution shift.

At the nominal point ($\sigma_q = 0.2L$, $\sigma_m = 0.05L$, $\rho = 0.9$), matched Bayes reduces the logical error rate after 50 rounds from 0.242 (NL) to 8.5×10^{-4}. The LSTM trained at that point reaches 6.6×10^{-3}. Being able to revise past decisions lowers the time-averaged error by a factor of about 3 (LSTM) to 7 (Bayes).

The matched filter is the best decoder at every test point, and a moderately mis-specified filter usually stays ahead of the LSTMs. The exception is weak correlation ($\rho = 0.3$–0.5), where the broadly trained LSTM beats the model-averaged filter. Matched LSTMs come within 2–17% of the filter where errors are common, and fall behind by 3–4 times where they are rare. Wrong assumptions hurt both: assuming strong correlation when there is none is worse than having no memory. Under regime-switching noise, a broadly trained LSTM beats every fixed-model filter, and only a regime-aware filter does better. We also report generalisation across noise strength, correlation and time, sensitivity to model mismatch, and compute cost.

Contents

1 Introduction **2**

2 Model **2**

3 Decoders **3**

4 Evaluation protocol **4**

5 Validation **5**

6 Results **6**

6.1 E1: main comparison at the nominal point 6

6.2 E2: stronger correlation 8

6.3 E3 and E4: stronger displacement and measurement noise 10

6.4 E5: Bayes with a mis-specified model 13
6.5 E6: regime-switching noise 14
6.6 E7: computational cost 15

7 Why: diagnostics **15**

8 Discussion: when does each method win? **17**

9 Limitations and future work **18**

A Reproducing the results **19**

1 Introduction

The Gottesman–Kitaev–Preskill (GKP) code stores a qubit in an oscillator and detects small displacements through syndromes that are periodic in the lattice spacing $L = \sqrt{\pi}$ [1, 2]. A single syndrome cannot distinguish a shift u from $u + L$, so a decoder that sees only the current round must guess. When the displacement noise is correlated in time, earlier syndromes carry information about the current error, and a decoder with memory can do better [3, 4].

The proposal asked four questions:

1. How much does memory help, comparing the nearest-lattice baseline, a sequential Bayesian filter and an LSTM?
2. How do the decoders behave as the noise becomes stronger or more correlated?
3. What happens when the Bayesian model is wrong, and when the LSTM sees noise it was not trained on?
4. When does each method work better, and why?

This report answers them with a simplified, validated simulator.

The reinforcement-learning (DQN) decoder and the experimental-data goal were explicitly left out of scope for this run (Section 9).

2 Model

Code and noise. We use dimensionless quadratures with $[\hat{q}, \hat{p}] = i$ and the square GKP code. Its stabilizer $S_q = e^{2i\sqrt{\pi}\hat{q}}$ fixes the lattice spacing $L = \sqrt{\pi}$. A shift by L is the logical $\bar{X}$ and a shift by $2L$ is a stabilizer. As in the proposal, we model only the q quadrature and simulate classical shifts rather than the oscillator state. For pure displacement noise this reproduces the ideal-code quantum dynamics; the simulator itself is validated separately in `docs/sim_validation.pdf`. In round $t = 1, \ldots, 50$ of a trial,

$$x \leftarrow x + \varepsilon_t, \qquad s_t = \mathrm{wrap}(x + \nu_t) \in [-L/2, L/2), \qquad x \leftarrow x - c_t,$$

where c_t is the decoder's correction. The displacement ε_t follows a stationary AR(1) process, $\varepsilon_t = \rho\varepsilon_{t-1} + \eta_t$ with $\mathrm{Var}\,\varepsilon_t = \sigma_q^2$, and $\nu_t \sim \mathcal{N}(0, \sigma_m^2)$ is i.i.d. measurement noise. The nominal point is $\sigma_q = 0.2L$, $\sigma_m = 0.05L$, $\rho = 0.9$. After round t the qubit is in a logical error when $\mathrm{round}(x/L)$ is odd. The *logical error rate* $P_L(n)$ is the fraction of trials in error after round n.

The decoding problem. Every decoder in this study applies $c_t = s_t + j_t L$ with $j_t \in \{0, 1\}$; the nearest-lattice (NL) decoder always plays $j_t = 0$. Under this restriction the residual after each round is an integer multiple of L minus ν_t, so

$$s_t = \mathrm{wrap}(z_t), \qquad z_t = \varepsilon_t + \nu_t - \nu_{t-1}, \tag{1}$$

whatever the decoder did earlier. Let $y_t = \mathrm{round}(z_t/L) \bmod 2$ be the parity of the NL decoder's jump in round t, and $K_t = \sum_{\tau \le t} y_\tau \bmod 2$ the NL decoder's logical state. Then any decoder's logical state after round t is

$$K_t \oplus J_t, \qquad J_t = \sum_{\tau \le t} j_\tau \bmod 2. \tag{2}$$

Decoding therefore amounts to estimating the parity K_t from the wrapped history $s_1, \ldots, s_t$. The best decoder plays $J_t = \hat{K}_t$, the MAP estimate given all syndromes so far. Because later syndromes do not depend on the j's, this greedy rule minimises $P_L(t)$ for every t at once, and it lets a decoder *revise* an earlier decision once later syndromes show it was wrong. A test in `tests/test_core.py` checks (1)–(2) against the simulator for an arbitrary decision sequence.

Two consequences shape the study:

- Every decoder sees the same syndromes on the same noise, so all comparisons are paired on common random numbers.
- Supervised labels for the LSTM (y_t and K_t from an NL run of the simulator) do not depend on the LSTM's own decisions, so training data and closed-loop testing are not mismatched.

Continuous fine corrections (estimating ν_t) are deliberately excluded.

3 Decoders

Nearest lattice (NL). $c_t = s_t$, i.e. $j_t = 0$. Uses only the current syndrome.

Bayesian grid filter. The hidden state is $(X_t \bmod 2L,\ \varepsilon_t)$, where $X_t = \sum_{\tau \le t} \varepsilon_\tau$ is the cumulative uncorrected displacement. The decoder knows its cumulative correction C_{t-1}, and $s_t = \mathrm{wrap}(X_t - C_{t-1} + \nu_t)$. This is an exact hidden Markov model of the simulator. On a grid with spacing h, each round performs:

1. A bin-integrated Gaussian AR(1) prediction of ε_t (one matrix product). The innovation variance is floored at $h^2/12$.
2. A shear $X_t = X_{t-1} + \varepsilon_t$ (an integer index shift per ε row).
3. Multiplication by the periodic likelihood $\sum_m \phi_\sigma\big(s_t + C_{t-1} - X + mL\big)$, evaluated analytically with $\sigma^2 = \sigma_m^2 + h^2/12$.
4. Normalisation.

The correction class j_t is the one whose half of the $2L$ circle around $s_t + C_{t-1} + j_t L$ carries more posterior mass. This is the cumulative MAP rule above. The filter is batched over trials on the GPU, and its grid convergence is shown in Table 1.

We use the filter with several assumed models:

- *Bayes (matched)* uses the true parameters. It is the syndrome-only optimum up to grid error.
- *Bayes (nominal)* always assumes the nominal parameters, which makes it the counterpart of an LSTM trained only at the nominal point.

- *Bayes-broad* is a Bayesian model average over 8 filters with $\rho' \in \{0, 0.6, 0.85, 0.95\}$, $\sigma'_q \in \{0.15, 0.25\}L$ and a fixed $\sigma'_m = 0.05L$. The filters start with equal weights, which are then updated by their marginal likelihood. It is the model-based counterpart of the broadly trained LSTM, except that it does not span the LSTM's σ_m range.
- *Bayes (regime-aware)* adds the hidden correlation regime to the state, for the switching-noise experiment.
- *Bayes (per-round)* is an ablation that, after each decision, discards the posterior mass of the rejected class, so it never revises a past decision.

LSTM. A two-layer LSTM (hidden size 128) reads $[s_t/L,\ \sin(2\pi s_t/L),\ \cos(2\pi s_t/L)]$ and has five sigmoid heads. Head "k" predicts the window parity $W_t^{(k)} = y_{t-k} \oplus \cdots \oplus y_t$ for $k \in \{0, 1, 2, 4\}$, and head "full" predicts K_t. A head's prediction is turned into actions through $J_t = \hat{W}_t^{(k)} \oplus J_{t-k-1}$, with $J_t = \hat{K}_t$ for the full head. The $k = 0$ head is the per-round (no-revision) ablation.

Training uses 200,000 sequences of 50 rounds and binary cross-entropy averaged over all heads, rounds and sequences (equal head weights), with AdamW, a cosine schedule, at most 40 epochs, and early stopping on validation loss. The head used at test time is the one with the lowest validation time-averaged logical error, and test data is never used for any choice. The training sets are:

- *nominal*: the nominal point only, 3 seeds;
- *broad*: per-sequence $\rho \sim U(0, 0.97)$, $\sigma_q \sim U(0.12, 0.28)L$, $\sigma_m \sim U(0.02, 0.08)L$, 3 seeds;
- *matched*: two single-point models at $\rho = 0.5$ and at $\sigma_q = 0.26L$, used to measure the generalisation gap;
- *switching*: the switching-noise model of E6.

Labels come from the true errors through an NL run (proposal, Sec. 5). At test time the LSTM sees only syndromes.

4 Evaluation protocol

- **Shared test noise.** At each test point all decoders run on the same test noise. Random streams are derived from a `SeedSequence` keyed by (split, experiment, point, chunk), so train, validation, development and test streams are disjoint by construction. A test checks, on sample keys, that test streams never coincide with any other split.
- **Test-set size.** The size is set once per point: 20,000-trial chunks are added until the reference filter (matched Bayes; the regime-aware filter in E6) shows at least 50 errors at round 50, or a cap is reached. The caps are 100,000 trials for E1, 60,000 for E6, 40,000 for E2, and 20,000 for E3 and E4. E5 uses a fixed 40,000 trials.
- **Reported quantities.** We report $P_L(10)$ and $P_L(50)$ with Wilson 95% intervals, plus the time-averaged logical error $\frac{1}{50}\sum_{n=1}^{50} P_L(n)$. When no errors are observed we report the 95% upper bound $3/N$.
- **Paired differences.** Differences between decoders are means of per-trial differences, with their standard errors.
- **LSTM seeds.** LSTM results are averaged over seeds, and the spread across seeds is given separately from the test-noise error.

- **In or out of distribution.** Points outside the broad training box are marked (hollow markers in figures, italics in tables).

5 Validation

Simulator. The simulator was validated before this study (`docs/sim_validation.pdf`). It reproduces:

- the exact single-round logical error of the square GKP code, including the value 0.110 at $\sigma = 0.555$ quoted by Gottesman, Kitaev and Preskill [1];
- the per-round jump statistics, which in theory do not depend on ρ (checked numerically at $\rho = 0$ and $\rho = 0.9$);
- the multi-round closed forms at $\rho = 0$ and $\rho \to 1$.

The validation script is re-run as experiment E0 (`results/E0_validate_sim.txt`).

Decoder correctness. The unit tests in `tests/test_core.py` check:

- the reduction (1)–(2);
- that "cheat" decoders given the true labels reach exactly zero logical error for every LSTM head;
- that the closed-loop LSTM decoder reproduces open-loop inference bit for bit;
- that the Bayes filter stays finite under severe model mismatch;
- that test-split random streams never coincide with the training, validation or development streams (all splits are disjoint by construction).

The filter's class probabilities are also approximately calibrated (Section 7).

Grid convergence. Table 1 compares grid resolutions on the same noise. With $n_x = 96$ points over $2L$, which is the resolution used throughout except for an assumed $\sigma'_m < 0.035L$ (where $n_x = 192$), the decisions differ from $n_x = 192$ in at most 10^{-3} of trial-rounds, and every summary statistic agrees to within its sampling error.

Case	n_x	mean error	$P_L(50)$	disagreement vs n_x=192
nominal	192	1.42×10^{-3}	1.05×10^{-3}	0
nominal	128	1.42×10^{-3}	1.05×10^{-3}	3×10^{-6}
nominal	96	1.42×10^{-3}	1.05×10^{-3}	3×10^{-6}
nominal	64	1.42×10^{-3}	1.05×10^{-3}	6×10^{-6}
sm0.02 rho0.99	192	4.09×10^{-3}	0	0
sm0.02 rho0.99	128	4.11×10^{-3}	0	3×10^{-5}
sm0.02 rho0.99	96	4.11×10^{-3}	0	8×10^{-5}
sm0.02 rho0.99	64	4.12×10^{-3}	0	2×10^{-4}
sq0.30	192	0.0322	0.0330	0
sq0.30	128	0.0322	0.0329	5×10^{-5}
sq0.30	96	0.0322	0.0329	9×10^{-5}
sq0.30	64	0.0322	0.0330	2×10^{-4}
rho0	192	0.2739	0.4191	0
rho0	128	0.2739	0.4194	5×10^{-4}
rho0	96	0.2738	0.4186	9×10^{-4}
rho0	64	0.2736	0.4187	2×10^{-3}

Table 1: E0: grid convergence of the Bayes filter (matched model, 20,000 trials of the development split).

6 Results

Unless stated otherwise, "error" means the time-averaged logical error over rounds 1–50. It is the most precise paired statistic and does not saturate. $P_L(50)$ is reported alongside it. A dagger (†) marks fewer than 10 observed errors, and "$< x$" is the 95% upper bound $3/N$ when none were observed.

6.1 E1: main comparison at the nominal point

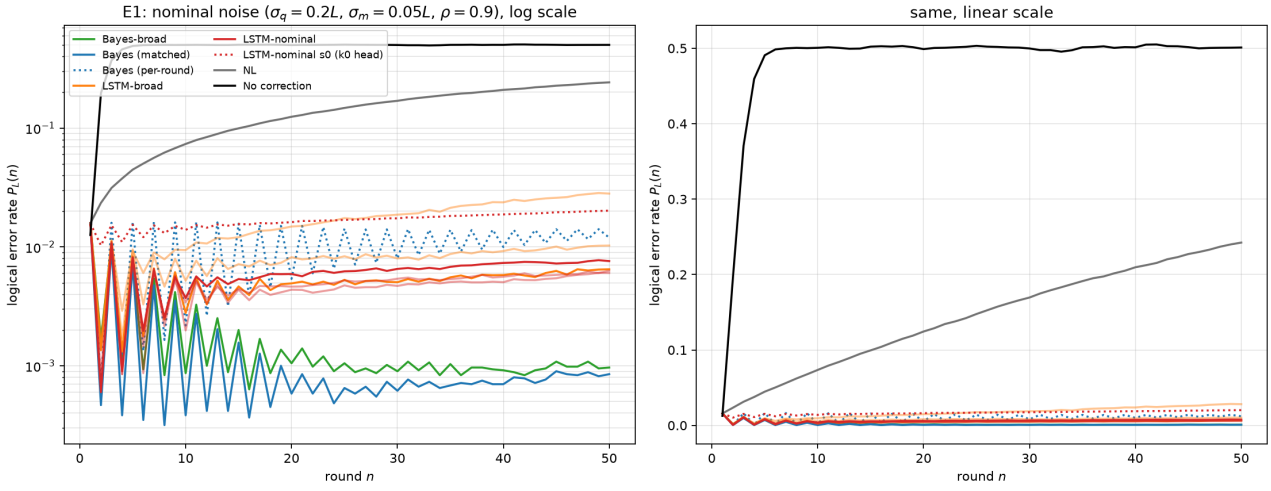


Figure 1: E1: logical error rate after each round at $\sigma_q = 0.2L$, $\sigma_m = 0.05L$, $\rho = 0.9$ (60,000 trials, common noise). Faint lines are the individual LSTM seeds.

Decoder	$P_L(10)$	$P_L(50)$	time-avg. error	paired diff. vs Bayes
No correction	0.501	0.501	0.481	$+0.480$ ($\pm 1\times10^{-3}$)
NL	0.073	0.242	0.144	$+0.142$ ($\pm 2\times10^{-3}$)
Bayes (matched)	3.8×10^{-4}	8.5×10^{-4}	1.5×10^{-3}	–
Bayes (per-round)	2.2×10^{-3}	0.012	0.011	$+9.2\times10^{-3}$ ($\pm 6\times10^{-4}$)
Bayes-broad	8.7×10^{-4}	9.7×10^{-4}	1.9×10^{-3}	$+3.7\times10^{-4}$ ($\pm 1\times10^{-4}$)
LSTM-nominal (mean of seeds)	3.0×10^{-3}	6.6×10^{-3}	5.5×10^{-3}	$+4.0\times10^{-3}$ ($\pm 4\times10^{-4}$)
LSTM-nominal s0	3.7×10^{-3}	7.6×10^{-3}	6.3×10^{-3}	$+4.8\times10^{-3}$ ($\pm 4\times10^{-4}$)
LSTM-nominal s1	3.4×10^{-3}	6.3×10^{-3}	5.3×10^{-3}	$+3.8\times10^{-3}$ ($\pm 4\times10^{-4}$)
LSTM-nominal s2	2.0×10^{-3}	6.0×10^{-3}	4.9×10^{-3}	$+3.4\times10^{-3}$ ($\pm 4\times10^{-4}$)
LSTM-nominal s0 (per-round head)	0.014	0.020	0.017	$+0.015$ ($\pm 9\times10^{-4}$)
LSTM-broad (mean of seeds)	5.8×10^{-3}	0.015	0.010	$+8.7\times10^{-3}$ ($\pm 4\times10^{-4}$)
LSTM-broad s0	2.8×10^{-3}	6.5×10^{-3}	5.4×10^{-3}	$+3.9\times10^{-3}$ ($\pm 4\times10^{-4}$)
LSTM-broad s1	9.4×10^{-3}	0.028	0.017	$+0.016$ ($\pm 7\times10^{-4}$)
LSTM-broad s2	5.2×10^{-3}	0.010	8.2×10^{-3}	$+6.6\times10^{-3}$ ($\pm 4\times10^{-4}$)

Table 2: E1 at the nominal point. The paired difference is the mean per-trial difference in time-averaged error from matched Bayes, with its 2-SE uncertainty. For the seed means the uncertainty shown is that of seed 0.

Memory changes the picture completely (Figure 1, Table 2):

- **Nearest lattice** accumulates errors steadily and reaches $P_L(50) = 0.242$.
- **Matched Bayes** reaches $P_L(50) = 8.5\times10^{-4}$, which is 51 errors in 60,000 trials. Its time-averaged error is 95 times smaller than NL's.
- **Bayes-broad** gets within about 25% of matched Bayes without knowing the parameters (1.9×10^{-3} against 1.5×10^{-3}).
- **LSTM-nominal** is trained at this exact point. It reaches a time-averaged error of 5.5×10^{-3}, which is 3.7 times that of matched Bayes but still about 26 times better than NL. The three seeds lie within about 30% of each other (3.3–4.2 times matched Bayes), a much smaller spread than for the broad seeds.
- **LSTM-broad** reaches 0.010 on average, with a large spread between seeds: seed 0 matches LSTM-nominal and seed 1 is 3 times worse. All three broad models finished their full 40-epoch schedule, and their validation losses differ little (Table 8). The spread therefore reflects how differently models trained on the whole box handle this particular corner of it, not undertraining.

Both ablations show that revising past decisions matters. Bayes restricted to per-round decisions has error 0.011, about 7 times worse than full Bayes. The LSTM's per-round head ($k = 0$) has error 0.017, about 2.7 times worse than its cumulative head. Section 7 explains the zig-zag in the early rounds.

6.2 E2: stronger correlation

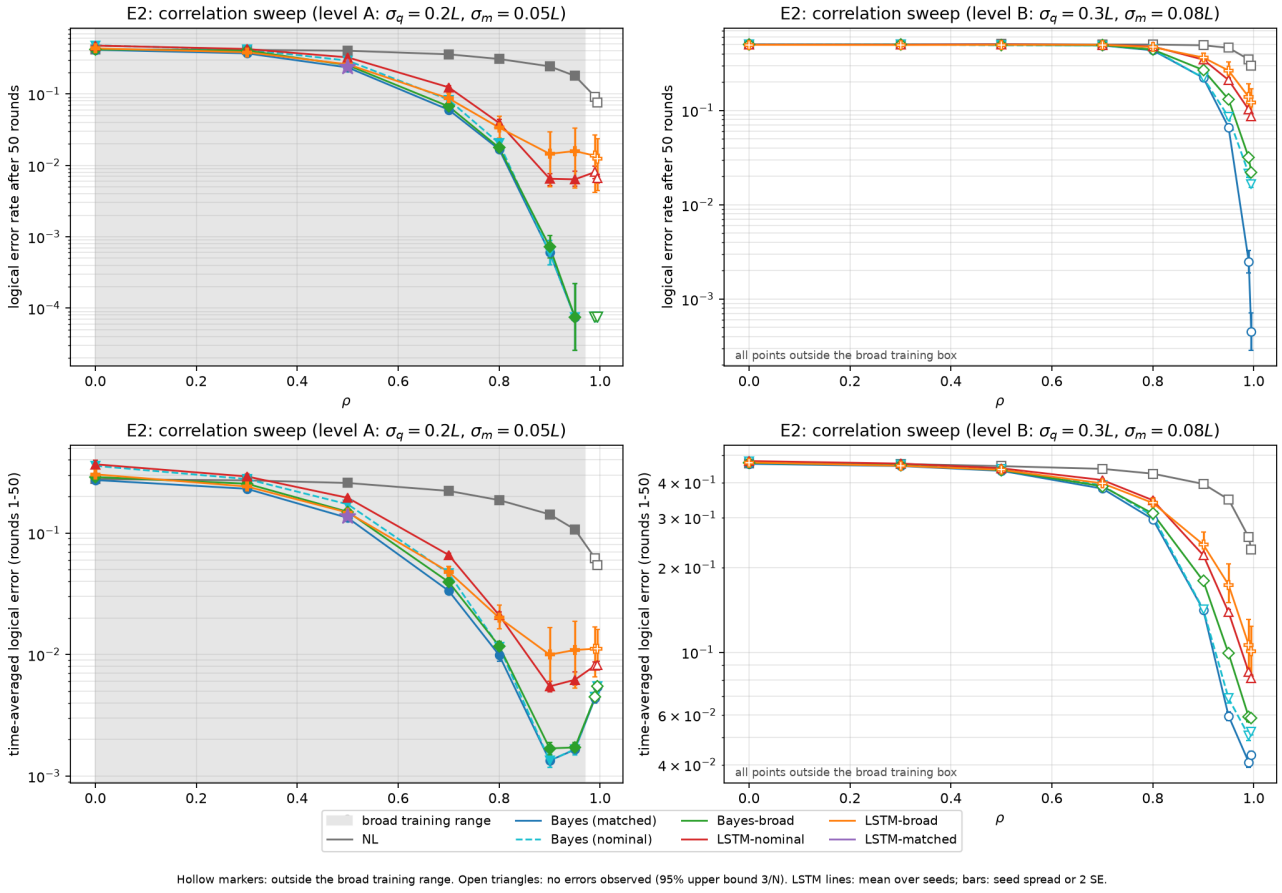


Figure 2: E2: correlation sweep at two noise levels. Top: $P_L(50)$. Bottom: time-averaged error. The shaded band is the ρ range of the broad training box. At level B, $\sigma_q = 0.3L$ lies outside the box ($\sigma_m = 0.08L$ sits at its upper edge), so every point there is out of distribution for both broad decoders.

ρ	NL	Bayes (matched)	Bayes (nominal)	Bayes-broad	LSTM-nominal	LSTM-broad	LSTM-matched
0.0	0.278	0.273	0.357	0.289	0.368	0.304	–
0.3	0.271	0.231	0.278	0.255	0.292	0.241	–
0.5	0.258	0.133	0.172	0.150	0.195	0.147	0.136
0.7	0.222	0.033	0.047	0.040	0.066	0.048	–
0.8	0.187	9.9×10^{-3}	0.012	0.012	0.021	0.020	–
0.9	0.142	1.3×10^{-3}	1.3×10^{-3}	1.7×10^{-3}	5.5×10^{-3}	10.0×10^{-3}	–
0.95	0.107	1.6×10^{-3}	1.7×10^{-3}	1.7×10^{-3}	6.2×10^{-3}	0.011	–
0.99	0.062	4.4×10^{-3}	4.5×10^{-3}	4.5×10^{-3}	8.1×10^{-3}	0.011	–
0.995	0.054	5.4×10^{-3}	5.5×10^{-3}	5.5×10^{-3}	8.2×10^{-3}	0.011	–

ρ	NL	Bayes (matched)	Bayes (nominal)	Bayes-broad	LSTM-nominal	LSTM-broad	LSTM-matched
0.0	0.468	0.466	0.475	0.469	0.476	0.471	–
0.3	0.465	0.457	0.462	0.460	0.467	0.458	–
0.5	0.457	0.440	0.446	0.443	0.449	0.443	–
0.7	0.447	0.380	0.390	0.388	0.408	0.397	–
0.8	0.430	0.296	0.304	0.310	0.346	0.338	–
0.9	0.395	0.141	0.142	0.179	0.221	0.241	–
0.95	0.348	0.060	0.069	0.099	0.139	0.173	–
0.99	0.256	0.041	0.051	0.059	0.085	0.106	–
0.995	0.231	0.043	0.052	0.058	0.081	0.101	–

Table 3: E2: time-averaged logical error. Top: level A ($\sigma_q = 0.2L$, $\sigma_m = 0.05L$). Bottom: level B ($\sigma_q = 0.3L$, $\sigma_m = 0.08L$). Italics mark points outside the broad training box.

Memory pays off more as correlation grows, up to $\rho \approx 0.9$. At level A, matched Bayes improves on NL by a factor of 1.02 at $\rho = 0$, 6.64 at $\rho = 0.7$ and 106 at $\rho = 0.9$. (Each value is NL's error divided by matched Bayes's error at that ρ.) Beyond $\rho = 0.9$ the factor shrinks (64.9 at $\rho = 0.95$, 10 at $\rho = 0.995$), because NL's error keeps falling while matched Bayes's error rises slightly. The small gain at $\rho = 0$ comes from the measurement-noise correlation in (1): z_t and z_{t+1} share ν_t.

At very high correlation ($\rho \geq 0.99$), Bayes shows no errors after round 50 ($P_L(50)$ is $< 7.5\times10^{-5}$), yet its time-averaged error rises again, to 4.4×10^{-3}. These errors are not confined to the first rounds. In about 1% of trials the logical state is wrong after every odd round and right after every even round, all the way to round 49. $P_L(50)$ is zero only because round 50 is even. Section 7 explains this odd/even pattern.

Wrong memory is worse than no memory. At $\rho = 0$ every decoder that assumes strong correlation does worse than the memoryless baseline:

- Bayes (nominal), which assumes $\rho = 0.9$, has error 0.357, against 0.278 for NL.
- LSTM-nominal has error 0.368.

The broad decoders, which have seen low-ρ noise, stay close to NL there (Bayes-broad 0.289, LSTM-broad 0.304). Even so, neither matches the matched filter's 0.273.

A matched LSTM is nearly optimal at moderate correlation. At $\rho = 0.5$, the LSTM trained at exactly that point reaches 0.136, which is only 1.02× matched Bayes. LSTM-nominal at the same point (out of distribution) has error 0.195.

The ML gap tends to grow as errors become rarer. Broadly, LSTM-nominal's error relative to matched Bayes grows as Bayes's own error falls, though not monotonically. Relative to matched Bayes, LSTM-nominal's error is

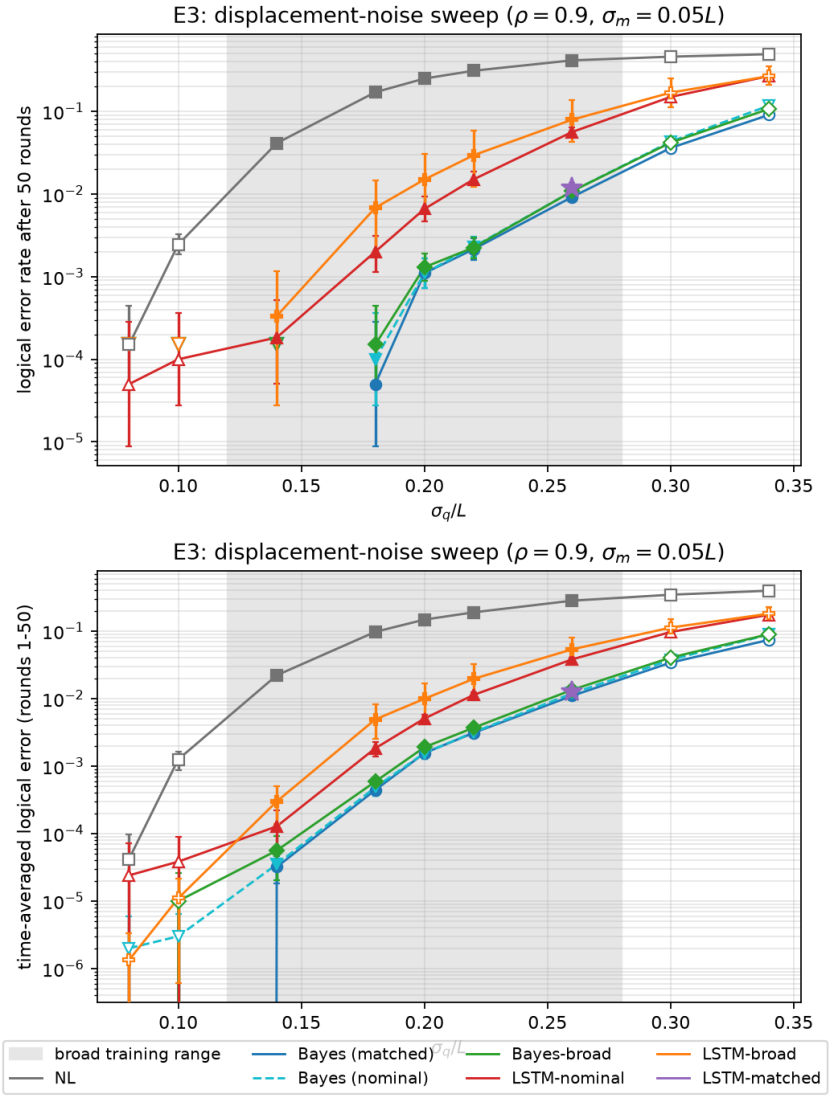
- 4.08× at $\rho = 0.9$ on level A, where Bayes has error 1.3×10^{-3};

- 1.56× at $\rho = 0.9$ on level B, where Bayes has error 0.141.

The ratio is lower at points where correlation is very high and errors are more common (A, $\rho = 0.99$: 1.85×), and higher at B, $\rho = 0.95$ (2.33×).

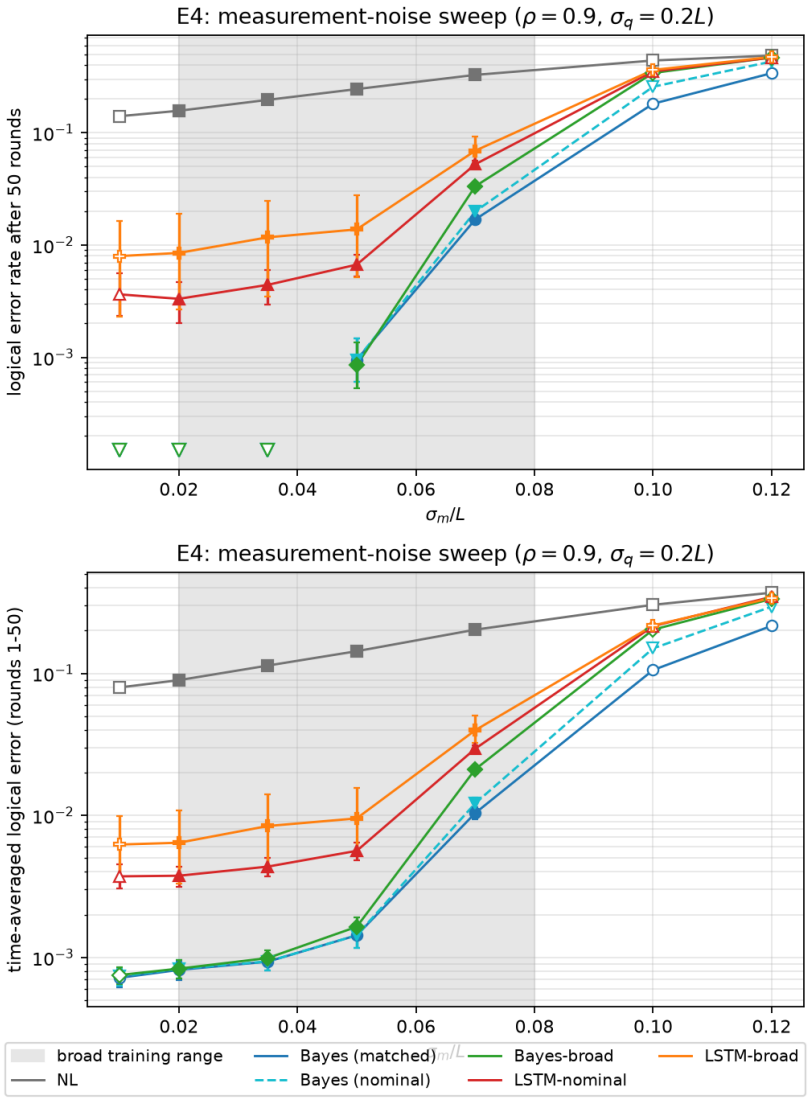
At level B the broad decoders are out of distribution in σ_q, and at the edge of their training range in σ_m. For $\rho \geq 0.8$ at level B, Bayes-broad does worse than Bayes (nominal), e.g. 0.179 against 0.142 at $\rho = 0.9$. For $\rho \leq 0.7$ it is slightly better. Both assume $\sigma'_m = 0.05L$, so the gap comes from Bayes-broad's (ρ', σ'_q) grid ($\sigma'_q \leq 0.25L$, against the true $0.3L$).

6.3 E3 and E4: stronger displacement and measurement noise



Hollow markers: outside the broad training range. Open triangles: no errors observed (95% upper bound 3/N). LSTM lines: mean over seeds; bars: seed spread or 2 SE.

Figure 3: E3: displacement-noise sweep at $\rho = 0.9$, $\sigma_m = 0.05L$.



Hollow markers: outside the broad training range. Open triangles: no errors observed (95% upper bound 3/N). LSTM lines: mean over seeds; bars: seed spread or 2 SE.

Figure 4: E4: measurement-noise sweep at $\rho = 0.9$, $\sigma_q = 0.2L$.

σ_q/L	NL	Bayes (matched)	Bayes (nominal)	Bayes-broad	LSTM-nominal	LSTM-broad	LSTM-matched
0.08	$1.5\times10^{-4\dagger}$	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	$5.0\times10^{-5\dagger}$	$<1.5\times10^{-4}$	–
0.1	2.4×10^{-3}	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	$1.0\times10^{-4\dagger}$	$<1.5\times10^{-4}$	–
0.14	0.041	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	1.8×10^{-4}	3.3×10^{-4}	–
0.18	0.170	$5.0\times10^{-5\dagger}$	$1.0\times10^{-4\dagger}$	$1.5\times10^{-4\dagger}$	2.0×10^{-3}	6.8×10^{-3}	–
0.2	0.249	1.1×10^{-3}	1.1×10^{-3}	1.3×10^{-3}	6.6×10^{-3}	0.015	–
0.22	0.310	2.1×10^{-3}	2.3×10^{-3}	2.2×10^{-3}	0.015	0.029	–
0.26	0.414	9.2×10^{-3}	0.011	0.011	0.056	0.079	0.012
0.3	0.459	0.036	0.043	0.042	0.150	0.168	–
0.34	0.492	0.091	0.117	0.107	0.268	0.267	–

σ_m/L	NL	Bayes (matched)	Bayes (nominal)	Bayes-broad	LSTM-nominal	LSTM-broad
0.01	0.140	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	3.6×10^{-3}	8.0×10^{-3}
0.02	0.157	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	3.3×10^{-3}	8.5×10^{-3}
0.035	0.196	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	$<1.5\times10^{-4}$	4.4×10^{-3}	0.012
0.05	0.244	9.5×10^{-4}	9.5×10^{-4}	8.5×10^{-4}	6.7×10^{-3}	0.014
0.07	0.327	0.017	0.020	0.033	0.052	0.069
0.1	0.440	0.181	0.257	0.340	0.351	0.362
0.12	0.487	0.339	0.431	0.468	0.467	0.473

Table 4: E3 (top) and E4 (bottom): $P_L(50)$; 20,000 trials per point.

Displacement noise. All errors rise steeply with σ_q (Figure 3). Wherever errors are resolved ($\sigma_q \geq 0.14L$), the ordering of the nominal and broad decoders never changes: matched Bayes is best, then Bayes (nominal) and Bayes-broad, then LSTM-nominal and LSTM-broad, then NL. The exception is the LSTM trained at $\sigma_q = 0.26L$, which is level with the model-based filters at that point. The model-based filters are remarkably insensitive to a wrong σ_q here. Bayes (nominal) assumes $\sigma_q = 0.2L$ and is within 20% of matched Bayes up to $\sigma_q = 0.34L$.

At $\sigma_q = 0.26L$ the matched LSTM (error 0.013) is again close to matched Bayes (1.17×). LSTM-nominal is 3.47× matched Bayes at that point.

Measurement noise. Measurement noise is the harder axis (Figure 4). Once σ_m reaches about $0.07L$, the syndrome no longer pins down the error well, and every decoder degrades quickly. At $\sigma_m = 0.10L$:

- Bayes (nominal), which assumes $\sigma_m = 0.05L$, has error 0.150, against 0.105 for matched Bayes.
- Bayes-broad, whose components all assume $\sigma'_m = 0.05L$, has error 0.203. That is still about 6% better than the LSTMs (LSTM-broad 0.217, LSTM-nominal 0.215), but most of its earlier lead is gone.

6.4 E5: Bayes with a mis-specified model

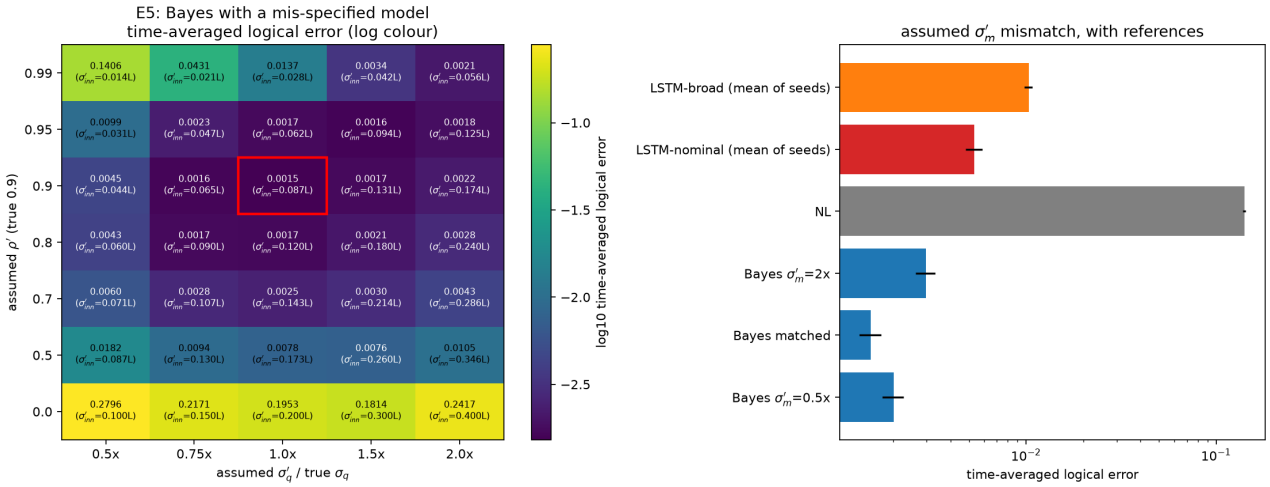


Figure 5: E5: time-averaged error of Bayes filters with wrong assumed parameters, at the nominal truth (40,000 trials, common noise). Each cell gives the innovation standard deviation implied by the assumed parameters; the true value is $0.087L$ (red box).

$\rho' \setminus \sigma'_q/\sigma_q$	0.5	0.75	1.0	1.5	2.0
0.0	0.280	0.217	0.195	0.181	0.242
0.5	0.018	9.4×10^{-3}	7.8×10^{-3}	7.6×10^{-3}	0.010
0.7	6.0×10^{-3}	2.8×10^{-3}	2.5×10^{-3}	3.0×10^{-3}	4.3×10^{-3}
0.8	4.3×10^{-3}	1.7×10^{-3}	1.7×10^{-3}	2.1×10^{-3}	2.8×10^{-3}
0.9	4.5×10^{-3}	1.6×10^{-3}	1.5×10^{-3}	1.7×10^{-3}	2.2×10^{-3}
0.95	9.9×10^{-3}	2.3×10^{-3}	1.7×10^{-3}	1.6×10^{-3}	1.8×10^{-3}
0.99	0.141	0.043	0.014	3.4×10^{-3}	2.1×10^{-3}

Table 5: E5: time-averaged error against the assumed ρ' (rows) and σ'_q/σ_q (columns). The truth is $\rho = 0.9$; the matched cell is in bold.

The filter tolerates moderate mismatch well. Every model with $0.8 \le \rho' \le 0.95$ and $0.75 \le \sigma'_q/\sigma_q \le 1.5$ stays within $1.6\times$ of the matched error. Near the true correlation, the assumed innovation standard deviation $\sigma'_q\sqrt{1-\rho'^2}$ (printed in each cell) is a useful guide, but it does not determine the error by itself:

- **Overestimating it by raising σ'_q is cheap.** With $\sigma'_q = 2\sigma_q$ the error only rises to 2.2×10^{-3}. Getting a similar overestimate by lowering ρ' costs much more: $\rho' = 0.5$ gives 7.8×10^{-3}.
- **Underestimating it is expensive.** A filter that thinks the displacement is more predictable than it is becomes overconfident. With $\rho' = 0.99$ the error is 0.014; with $\rho' = 0.99$ and $\sigma'_q = 0.5\sigma_q$ it collapses to NL's level (0.141).
- **Assuming no correlation ($\rho' = 0$)** is worse than NL (0.195 against 0.141). The filter then commits to the wrong cell whenever the current syndrome is ambiguous, and it never uses the history that would have resolved it.
- **A wrong measurement noise** costs up to about a factor of 2 at the two values tested ($\sigma'_m = 0.5\sigma_m$: 2.0×10^{-3}; $\sigma'_m = 2\sigma_m$: 3.0×10^{-3}).

Measured against this scale, LSTM-nominal (5.3×10^{-3}, at its own training point) is about as good as a Bayes filter with a moderately wrong model, e.g. $\rho' = 0.7$ and $\sigma'_q = 0.5\sigma_q$. Many mis-specified filters beat it, and some severely mis-specified ones do not.

6.5 E6: regime-switching noise

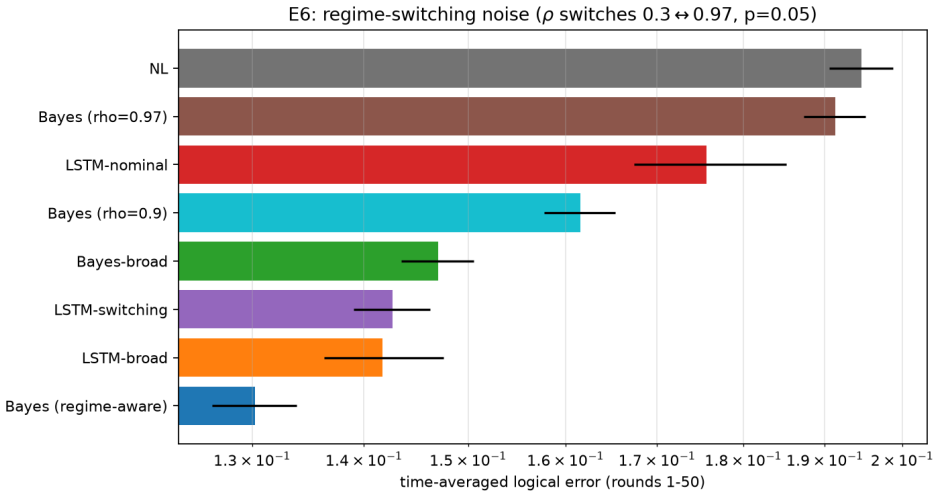


Figure 6: E6: the correlation switches between $\rho = 0.3$ and $\rho = 0.97$ with probability 0.05 per round. Bars show the 2-SE test-noise uncertainty plus, for the LSTM seed means, the spread between seeds.

Decoder	$P_L(10)$	$P_L(50)$	time-avg. error
NL	0.101	0.320	0.195
Bayes (regime-aware)	0.061	0.226	0.130
Bayes (rho=0.9)	0.081	0.269	0.162
Bayes (rho=0.97)	0.099	0.312	0.191
Bayes-broad	0.065	0.251	0.147
LSTM-nominal (mean of seeds)	0.091	0.290	0.176 [0.17, 0.18]
LSTM-broad (mean of seeds)	0.065	0.246	0.142 [0.14, 0.14]
LSTM-switching	0.068	0.247	0.143

Table 6: E6: switching noise (20,000 trials, common noise).

Under noise whose correlation switches between regimes, the decoder that models the switching is best: regime-aware Bayes has error 0.130, against 0.195 for NL. This is the setting where the proposal expected machine learning to help when the Bayesian model is wrong, and it does. LSTM-broad (0.142) beats every *fixed-model* Bayes filter:

- $\rho = 0.9$: 0.162;
- $\rho = 0.97$: 0.191, worse than LSTM-nominal;
- Bayes-broad: 0.147.

An LSTM trained on the switching noise itself (0.143) is no better than the broad one. So the LSTM's advantage over the fixed filters here comes from having seen varied correlations, not from learning this specific process.

6.6 E7: computational cost

Decoder	device	batch	ms / round	trial-rounds / s
Bayes (matched)	cuda	1	3.43	292
Bayes (matched)	cuda	4096	13.3	3.08e+05
Bayes-broad	cuda	1	22.6	44.2
Bayes-broad	cuda	4096	122	3.36e+04
LSTM-nominal s0	cuda	1	1.5	668
LSTM-nominal s0	cuda	4096	13	3.15e+05
NL	cpu	1	0.00644	1.55e+05
NL	cpu	512	0.0188	2.72e+07
Bayes (matched)	cpu	1	0.873	1.15e+03
Bayes (matched)	cpu	512	53.1	9.65e+03
Bayes-broad	cpu	1	4.77	210
Bayes-broad	cpu	512	437	1.17e+03
LSTM-nominal s0	cpu	1	0.944	1.06e+03
LSTM-nominal s0	cpu	512	1.93	2.66e+05

Table 7: E7: decoding cost per round, measured on an otherwise idle NVIDIA GeForce RTX 5080 Laptop GPU and the host CPU. Batch 1 gives the latency of one decoder instance; the large batch gives throughput.

For a single trial on the CPU, NL takes microseconds, while matched Bayes and the LSTM both take about a millisecond per round (0.873 and 0.944 ms). Batched, the picture differs by device:

- **GPU:** the LSTM and the grid filter have similar throughput ($3.15e+05$ against $3.08e+05$ trial-rounds/s).
- **CPU:** the LSTM's throughput is about 27 times the filter's.

Batched, Bayes-broad (8 components) costs as much as 8–9 single filters; for a single trial it costs about 5.5–6.6 times one filter. Training each LSTM took 6–11 minutes of wall time for 40 epochs, measured while the GPU was shared with the Bayes evaluations (Table 8). The filter's cost grows with the grid ($n_x \times n_\varepsilon$ cells per trial) and would grow further with more hidden variables, e.g. for regimes or the p quadrature. The LSTM's cost is fixed by its size.

7 Why: diagnostics

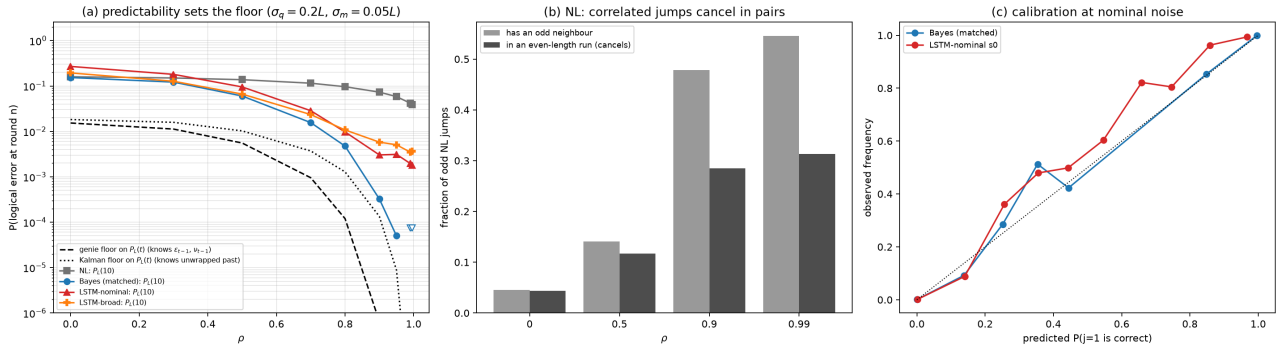


Figure 7: E8 diagnostics. (a) $P_L(10)$ against ρ (level A), with two analytic floors: a genie that knows ε_{t-1} and ν_{t-1}, and the optimal (Kalman) predictor given the *unwrapped* past. (b) NL's odd jumps cluster as correlation grows. (c) Calibration of the per-round probability $P(j_t = 1 \text{ is correct})$ at the nominal point.

Why memory helps: predictability. With $c_t = s_t + j_t L$, the only uncertainty in round t is which lattice cell $z_t = \varepsilon_t + \nu_t - \nu_{t-1}$ fell into (1). The history predicts ε_t with an innovation standard deviation of $\sigma_q\sqrt{1-\rho^2}$, which shrinks from σ_q at $\rho = 0$ to almost nothing at $\rho \to 1$. Figure 7(a) shows that the per-round floors follow this.

- A decoder that knew the unwrapped past would err in about 1.8% of rounds at $\rho = 0$, but only about 10^{-4} at $\rho = 0.9$.
- Matched Bayes tracks this floor at high ρ. At $\rho = 0.9$ its $P_L(10)$ is within a factor of about 3 of the Kalman floor, which knows strictly more than the decoder.
- At low ρ the gap is wide because errors accumulate over the 10 rounds.

Why NL also improves with ρ. NL's per-round jump probability does not depend on ρ (`docs/sim_validation`), yet its $P_L(50)$ falls as ρ grows. The reason is that correlated jumps come in runs (Figure 7(b)):

- at $\rho = 0$, 0.05 of odd jumps have an odd neighbour;
- at $\rho = 0.9$, 0.48 do, and 0.29 sit in even-length runs whose parities cancel.

This is the same effect reported in the simulator validation.

Why the early rounds zig-zag. When ρ is close to 1, a large ε near $\pm L/2$ is nearly frozen. The first syndrome cannot tell which cell it is in. If the decoder guesses wrong, the error then flips the logical state every round until the evidence (the slow drift of ε) resolves it. This is the odd/even pattern of Figure 1, and the $\rho \to 1$ limit derived in `docs/sim_validation`. Bayes resolves most such cases within a few rounds. At $\rho \geq 0.99$, however, about 1% of trials have a displacement so close to the cell boundary that the slow drift never settles it, and those trials stay wrong after every odd round. That is why $P_L(50)$ (an even round) vanishes there while the time-averaged error does not.

Why revising decisions matters. A decision that looked right given $s_1..s_t$ can be shown wrong by $s_{t+1}, s_{t+2}, \ldots$ once the drift of ε becomes visible. The cumulative rule $J_t = \hat{K}_t$ undoes such mistakes, while per-round rules lock them in. The ablations quantify this at the nominal point:

- Bayes: 1.5×10^{-3} (cumulative) against 0.011 (per-round);
- LSTM: 6.3×10^{-3} against 0.017.

In every LSTM, validation selected the cumulative head (Table 8).

Where the LSTM loses to Bayes. At the nominal point, the two decoders disagree in only 5.9×10^{-3} of trial-rounds. When they disagree, Bayes is right 98% of the time, and its posterior is almost always decisive there; it is the LSTM's output that sits close to 0.5. The LSTM's errors are therefore concentrated in rare rounds that the exact posterior settles clearly but the network has not learned: rounds with an odd nearest-lattice jump, where these decisions arise, are only about 2% of the training data.

This fits the pattern in E2 and E3: the LSTM approaches Bayes where errors are common (matched LSTM within 2–17% at $\rho = 0.5$ and at $\sigma_q = 0.26L$), and falls behind where they are rare. The LSTM's probabilities are also biased low (Figure 7(c)). From a predicted value of about 0.25 upwards, observed frequencies exceed its predicted $P(j_t = 1)$ by up to about 0.15, so it is under-confident above 0.5 and over-confident towards $j_t = 0$ between 0.25 and 0.5. Bayes is approximately calibrated.

Generalisation over time. Run for 200 rounds, all memory decoders keep working well past their 50-round training length (Figure 8). Errors accumulate slowly:

- matched Bayes: $9.5\times10^{-4} \to 2.6\times10^{-3}$;
- LSTM-nominal: $6.8\times10^{-3} \to 0.018$;
- NL: $0.240 \to 0.463$.

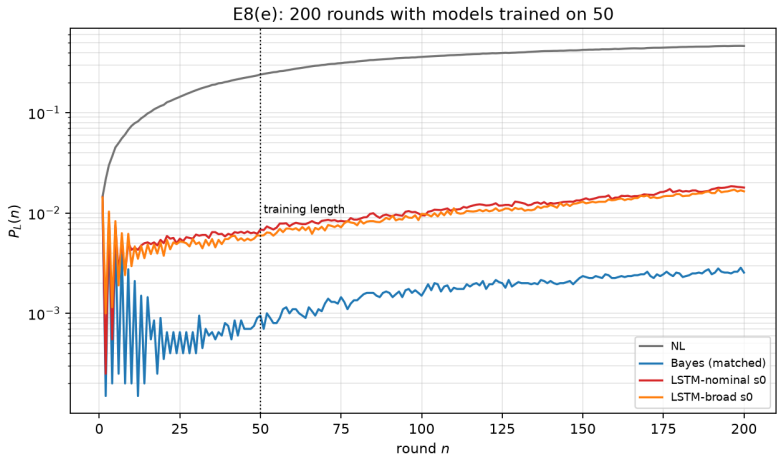


Figure 8: E8(e): 200 rounds at the nominal point; the LSTMs were trained on 50-round sequences.

8 Discussion: when does each method win?

Memory helps, and more so the more correlated the noise. Once $\rho \gtrsim 0.5$, every decoder in the parameter sweeps (E2–E4) beats nearest lattice, by up to two orders of magnitude at $\rho \approx 0.9$. Badly mis-specified filters (E5) are the exception. The gain follows the predictability of the next displacement, $\sigma_q\sqrt{1-\rho^2}$ (Section 7). Much of it, though not most, comes from being able to *revise* earlier decisions. At the nominal point, predicting the next displacement alone gives a factor of about 13 for Bayes (about 9 for the LSTM), and revision adds a further factor of about 7 (about 2.7 for the LSTM).

When the model is right, Bayes wins. With the correct noise model, the grid filter is the optimal syndrome-only decoder for this action set, and it was best at every test point. The best LSTMs come within 2–17% of it where logical errors are common (matched models at $\rho = 0.5$: 2%; at $\sigma_q = 0.26L$: 17%). They fall 3–4 times behind where errors are rare (the nominal point): the decisive rounds are rare in the training data, and the network has not learned them.

When the model is moderately wrong, Bayes usually still wins. A filter whose assumed parameters are moderately off loses little: every model with $0.8 \le \rho' \le 0.95$ and $0.75 \le \sigma_q'/\sigma_q \le 1.5$ stays within 1.6× of the matched error. Overestimating σ_q is cheap. Underestimating the innovation noise, lowering ρ' substantially, or ignoring the correlation is expensive, and can be worse than no memory at all (E5).

In the parameter sweeps, the fixed nominal filter stayed close to matched Bayes over a wide range of σ_q and for $\rho \ge 0.8$. Averaged over LSTM seeds, its error was at or below that of the nominally trained LSTM at every sweep point; at $\rho = 0$ the two were within the LSTM seed spread.

A Bayesian average over a coarse grid of models (Bayes-broad) matched or beat the broadly trained LSTM at most test points, including most of level B, where both are out of distribution. The exceptions are weak-to-moderate correlation inside the training box: the LSTM was better at level A with $\rho = 0.3$ for all three seeds, and slightly better at $\rho = 0.5$.

When the model is structurally wrong, ML helps. Under regime-switching correlation (E6), every fixed-model filter is mis-specified, and the LSTM trained on varied noise beat all of them. Only a filter that models the switching did better. This is the clearest support in our data for the proposal's hypothesis that machine learning helps when the Bayesian model is inaccurate. The advantage came from training on varied correlations rather than on the switching process itself.

Generalisation needs varied training.

- A network trained at one point degrades away from it. At $\rho = 0$ it is worse than NL on average over seeds, because it applies memory that is not there.
- Training on a broad box removes most of that failure, at the cost of some in-distribution performance and a larger spread between seeds.
- The same holds for Bayes: a single assumed model is fragile in the same places, and averaging over models removes most of the problem. At $\rho = 0$, Bayes-broad is still slightly worse than NL.

Cost. On a GPU, both memory decoders process about 3×10^5 trial-rounds per second, with a latency of a few milliseconds per round. On a CPU, the single-trial latency is similar (about 0.9 ms per round for both), but for batched decoding the LSTM's throughput is about 27 times the grid filter's. The filter's cost grows with the number of hidden variables, which favours learned decoders for richer noise models. The LSTM, in turn, needs training data and about 6–11 minutes of GPU time per model.

Answer to the research question. In this setting, machine learning did not outperform a correctly specified Bayesian filter, and it rarely outperformed a mildly mis-specified one. The exception is a model-averaged filter at weak correlation. A broadly trained LSTM is nevertheless a robust, cheap and assumption-light decoder:

- it beats nearest-lattice decoding by large factors under correlated noise;
- it beats mis-specified filters when the true noise is structurally different from what the filter assumes.

9 Limitations and future work

Model scope.

- *One quadrature, classical shifts.* Everything is in the q quadrature with classical shifts on an ideal (infinite-energy) code. Finite-energy envelopes, p-quadrature noise, photon loss and ancilla noise inside the syndrome-extraction circuit are not modelled. The finite-Δ state in `quantum/encoding.py` is the natural next step for checking the shift model.
- *Discrete corrections.* Decoders choose only the parity j_t of a correction $s_t + j_t L$. A decoder that also estimates ν_t and applies a continuous fine correction could reduce the residual further. This restriction is what makes the syndrome history independent of the decoder (Section 2), so it simplifies training and makes the comparisons paired, but it is a real limitation.

- *One family of correlated noise.* The noise is a stationary Gaussian AR(1), plus one two-regime switching variant. Heavier tails, drifts and non-stationary noise were not tested.

Decoders.

- *Architecture search.* The LSTM architecture and training recipe were fixed in advance and not tuned. A larger model, more data or a curriculum could close part of the gap to the Bayes filter.
- *Bayes-broad coverage.* Bayes-broad averages over 8 fixed AR(1) models with $\sigma'_m = 0.05L$. It therefore covers the broad box less densely than the broad LSTM's training data, in particular in σ_m.

Statistics.

- *Small error rates.* Where matched Bayes makes almost no errors (high ρ at the nominal noise level), $P_L(50)$ is only bounded from above. Such points are marked in the tables, and the comparisons there rely on the time-averaged error.
- *LSTM seeds.* LSTM-nominal and LSTM-broad have 3 seeds each, so their seed spread is only indicative. The matched and switching LSTMs were trained with a single seed, so their seed-to-seed variation is not measured.

Out of scope for this run.

- *Reinforcement learning.* The DQN decoder (proposal, item 4) was not built. With the parity action set used here, actions do not affect future observations, so the decoding problem is a contextual bandit with a known optimal policy (matched Bayes). An RL decoder would become more interesting with continuous corrections or with noise that reacts to the corrections.
- *Experimental data.* Testing on recorded GKP syndrome data (e.g. Campagne-Ibarcq *et al.* [6]) was not attempted.

Compute. The run used one laptop RTX 5080. LSTM training and the Bayes evaluations ran concurrently on it, so the timings in E7 were measured separately, after training finished.

A Reproducing the results

Everything lives on the local branch `experiment/decoder-comparison` (worktree `/home/babay/programs/QC/qkp-experiment`). From the repo root, with the `.venv` created by `uv` (CUDA 12.8 PyTorch):

```
QKP_MODE=quick .venv/bin/python -m pytest -q        # tests
.venv/bin/python -m training.train_lstm              # LSTMs -> checkpoints/
.venv/bin/python -m experiments.run_all              # E0-E8 -> results/cache/
.venv/bin/python -m experiments.analyze              # CSV, figures, summary.json
.venv/bin/python -m writeup.make_tables              # tables/macros for this report
cd writeup && xelatex report.tex && xelatex report.tex
```

`QKP_MODE=quick` runs the whole pipeline on small sizes, using only the development random streams and writing to `results/quick/`.

Model	chosen head	stop	epochs	minutes
lstm_broad_s0	full	max epochs	40	7.6
lstm_broad_s1	full	max epochs	40	11.2
lstm_broad_s2	full	max epochs	40	11.2
lstm_matched_rho0.5_s0	full	max epochs	40	11.1
lstm_matched_sq0.26_s0	full	early stop	38	10.7
lstm_nominal_s0	full	max epochs	40	5.6
lstm_nominal_s1	full	max epochs	40	6.2
lstm_nominal_s2	full	max epochs	40	6.7
lstm_switching_s0	full	max epochs	40	10.9

Table 8: LSTM training. Training stops at 40 epochs (cosine schedule), through early stopping, or at a 30-minute wall-clock guard. Models that hit an earlier 10-minute guard while sharing the GPU were retrained so that all final models finish their schedule. The head is chosen on validation data.

References

[1] D. Gottesman, A. Kitaev, J. Preskill, "Encoding a qubit in an oscillator," *Phys. Rev. A* **64**, 012310 (2001). https://arxiv.org/abs/quant-ph/0008040

[2] A. L. Grimsmo, S. Puri, "Quantum error correction with the Gottesman–Kitaev–Preskill code," *PRX Quantum* **2**, 020101 (2021). https://arxiv.org/abs/2106.12989

[3] K. H. Wan, A. Neville, S. Kolthammer, "Memory-assisted decoder for approximate Gottesman–Kitaev–Preskill codes," *Phys. Rev. Research* **2**, 043280 (2020). https://arxiv.org/abs/1912.00829

[4] M. Puviani *et al.*, "Non-Markovian feedback for optimized quantum error correction," *Phys. Rev. Lett.* **134**, 020601 (2025).

[5] K. Fukui, A. Tomita, A. Okamoto, K. Fujii, "High-threshold fault-tolerant quantum computation with analog quantum error correction," *Phys. Rev. X* **8**, 021054 (2018). https://arxiv.org/abs/1712.00294

[6] P. Campagne-Ibarcq *et al.*, "Quantum error correction of a qubit encoded in grid states of an oscillator," *Nature* **584**, 368–372 (2020).